

Documentación Técnica — Electronic Nose Project

Caso de uso

Sistema de nariz electrónica para clasificar vinos por calidad usando sensores MQ de resistencia química (chemoresistivos). El hardware mide cómo cambia la resistencia eléctrica de cada sensor al exponerse a los compuestos volátiles del vino. El software convierte esas curvas de resistencia en un vector de números y entrena un clasificador SVM para distinguir entre:

Clase	Significado
AQ	Vino de baja calidad (Acqua Quality)
LQ	Vino de calidad media (Low Quality)
HQ	Vino de alta calidad (High Quality)
ETH	Etanol puro (sustancia de control/referencia)

El array tiene 6 sensores en dos grupos (MQ3, MQ4, MQ6 × 2 réplicas), más lecturas de humedad y temperatura ambiente.

Flujo del pipeline



Las Fases 1-3 están implementadas en `phase_1_3_feature_extraction.py` y se ejecutan por cada archivo de medición dentro de la Fase 4.

`phase_1_3_feature_extraction.py` — Procesamiento de señal y extracción de features

Qué hace

Convierte un archivo `.txt` de medición bruta (una fila por muestra temporal, columnas = sensores) en un único vector de características numéricas listo para el modelo.

Por qué este enfoque

Los sensores MQ son resistivos: su resistencia varía según la concentración de gas. La curva temporal completa tiene forma de pico: sube durante la exposición al volátil y vuelve a bajar en la limpieza. No se puede usar directamente la serie temporal como entrada al clasificador porque:

- Cada medición puede tener distinto número de muestras
- El modelo necesita vectores de longitud fija
- La magnitud absoluta de resistencia varía entre sesiones y dispositivos

La solución es extraer **estadísticos resumidos** de segmentos temporales bien definidos.

Fase 2: Suavizado — Filtro Savitzky-Golay

Qué hace: suaviza la señal eliminando ruido de alta frecuencia.

Por qué Savitzky-Golay y no media móvil:

La media móvil simple promedia los últimos `w` valores, lo que aplana los picos. El filtro Savitzky-Golay ajusta un **polinomio de grado `p`** a una ventana deslizante de `w` puntos por mínimos cuadrados y devuelve el valor central del polinomio ajustado:

$$\hat{x}_i = \sum_{k=-m}^m c_k \cdot x_{i+k}$$

donde los coeficientes c_k se derivan de la pseudoinversa de la matriz de Vandermonde. Esto **preserva la forma de los picos y las pendientes** mejor que la media móvil, lo cual es crítico porque las features `max` y `slope` dependen directamente de ellos.

Parámetros usados: `window=15` muestras (impar, obligatorio), `polyorder=3`.

Limitación actual: la ventana es fija para todas las señales. Si una señal es muy corta (`len < window`), el código devuelve la señal sin filtrar, lo que puede introducir inconsistencias.

Fase 2: Normalización por línea base — cambio fraccional ($\Delta R/R_0$)

Qué hace: elimina el offset absoluto de cada sensor expresando la respuesta como desviación relativa respecto al estado en reposo.

Fórmula:

$$s_{norm}(t) = \frac{R_0 - R_s(t)}{R_0}$$

donde:

- R_0 = resistencia media en los primeros 2 segundos (línea base, sensor al aire limpio)
- $R_s(t)$ = resistencia en cada instante durante la exposición

Por qué esta fórmula:

Es la normalización estándar en sensores chemoresistivos (usada en la literatura de narices electrónicas desde los años 90). Tiene dos propiedades importantes:

- Elimina la variabilidad entre sensores:** dos sensores MQ3 con resistencias base distintas dan la misma salida si reaccionan igual.
- El signo indica dirección:** si $R_s < R_0$ (el gas reduce la resistencia), el valor es positivo. Si $R_s > R_0$, negativo. Esto es informativo para algunos gases.

Limitación actual: si la línea base $R_0 = 0$ (sensor defectuoso o saturado), se usa Z-score como fallback, lo cual no es físicamente equivalente. Debería marcarse ese archivo como inválido.

Fase 3: Extracción de características por ventanas temporales — dos modos

Qué hace: divide la señal normalizada en segmentos de tiempo y extrae 3 estadísticos de cada segmento × cada sensor.

Ventanas configuradas:

Ventana	Rango	Fase física
w0-2	0–2 s	Línea base (señal en reposo)
w2-10	2–10 s	Inyección del volátil (respuesta del sensor)
w10-20	10–20 s	Limpieza (retorno a la línea base)

Por qué segmentar en ventanas:

La respuesta completa del sensor contiene información en tres fases distintas. Tratarla como un todo perdería la estructura temporal:

- La pendiente en la fase de inyección refleja la **cinética de adsorción** del gas.
- El máximo refleja la **concentración de equilibrio** o saturación.
- La fase de limpieza refleja la **cinética de desorción**, que difiere según el compuesto.

Cada ventana aporta información complementaria; concatenarlas amplía la capacidad discriminativa del modelo.

Características extraídas por ventana:

1. **Máximo** $\max(s_{norm})$: pico de respuesta. Relacionado con la concentración máxima de analito que alcanza el sensor.
2. **Área bajo la curva (AUC)**:

$$AUC = \int_{t_{start}}^{t_{end}} s_{norm}(t) dt \approx \sum_i s_i \cdot \Delta t$$

Implementado con la regla del trapecio (`np.trapezoid`). Representa la **exposición total acumulada** al analito. Es robusto frente a ruido puntual porque integra toda la ventana.

3. **Pendiente máxima**:

$$slope = \max \left| \frac{ds_{norm}}{dt} \right| \approx \max \left| \frac{s_i - s_{i-1}}{\Delta t} \right|$$

Mide la **velocidad de respuesta máxima**. Diferentes volátiles y concentraciones producen distintas tasas de cambio en la resistencia.

Resultado (modo `handcrafted`): 6 sensores $\times$ 3 ventanas $\times$ 3 métricas = **54 características** por muestra.

Convención: `{sensor}_{ventana}_{métrica}` — e.g., `MQ3_1_w2-10_max` .

Fase 3 alternativa: extracción PCA por sensor (V3, modo `pca_signal`)

Qué hace: en lugar de calcular max/AUC/slope, proyecta cada segmento de señal sobre los componentes principales aprendidos del dataset completo.

Por qué PCA sobre la curva:

El enfoque manual presupone qué aspectos de la curva son discriminativos (el pico, el área, la pendiente). PCA deja que el dataset decida: busca las direcciones de máxima varianza en el espacio de los puntos de la curva. En dominios similares (Macías et al., 2013) el PC1 captura >98% de la varianza — toda la información discriminativa cabe en un número.

Formalmente, dado un conjunto de n segmentos de longitud p para una clave (sensor, ventana), se construye la matriz $X \in \mathbb{R}^{n \times p}$ y se calcula:

$$X = U \Sigma V^T \quad \Rightarrow \quad \text{score}_i = (x_i - \bar{x}) \cdot V_k$$

donde V_k son los k vectores propios seleccionados por umbral de varianza acumulada. El número de componentes k se elige automáticamente como el mínimo que satisface:

$$\frac{\sum_{j=1}^k \sigma_j^2}{\sum_{j=1}^p \sigma_j^2} \geq \theta \quad (\theta = 0.95 \text{ por defecto})$$

Un PCA por (sensor × ventana): cada sensor tiene respuesta química distinta y cada ventana captura una fase diferente del experimento. Mezclar sensores en un PCA global oscurecería esas diferencias. Con 6 sensores × 3 ventanas = **18 modelos PCA independientes**.

Resultado (modo `pca_signal`): número de features variable, determinado por el umbral de varianza. Típicamente 1-3 PCs por (sensor, ventana) → **18–54 características**, con la ventaja de que son las más informativas matemáticamente.

Convención: `{sensor}_{ventana}_pc{n}` — e.g., `MQ3_1_w2-10_pc1`.

Limitación conocida: el PCA se ajusta sobre todo el dataset en la Fase 4, antes del split train/test de la Fase 5. Esto introduce un leve data leakage en la transformación PCA (no en el clasificador). El paper de referencia aplica el mismo enfoque. Para eliminarlo completamente habría que integrar el PCA dentro del pipeline de Phase 5 como un paso previo al scaler.

Archivo generado: `data/processed/pca_transformers.pkl` — necesario junto a `best_model.pkl` para clasificar nuevas muestras.

`phase_4_dataset_generation.py` — Ensamblaje del dataset maestro

Qué hace

Itera sobre todos los archivos `.txt` de `data/raw/`, aplica la extracción de Fases 1-3 a cada uno, y agrega los resultados en un DataFrame de pandas que guarda como CSV.

Por qué este diseño

El dataset maestro es el contrato entre el procesamiento de señal y el modelo ML. Separarlo en un CSV permite:

- Reentrenar el modelo sin reprocesar todas las señales
- Inspeccionar y debuggear los datos intermedios
- Añadir más muestras incrementalmente

Etiquetado automático: la clase (`AQ`, `HQ`, `LQ`, `ETH`) se infiere del nombre del archivo mediante búsqueda de substring (e.g., `AQWINE` → `AQ`). Esto es frágil si los nombres de archivo no siguen la convención.

Escalabilidad de columnas: el `pd.DataFrame(self.records)` crea columnas automáticamente a partir de las keys del diccionario de features. Si se añaden nuevas ventanas o métricas en la Fase 3, el CSV se expande

automáticamente sin cambiar la Fase 4.

Limitación actual: no hay control de versiones del dataset. Si se mezclan archivos procesados con distintas configuraciones (p.ej., distintas ventanas temporales), el CSV tendrá columnas inconsistentes con `NaN`.

phase_5_model_training.py — Entrenamiento del modelo SVM

Qué hace

Carga el dataset maestro, entrena un SVM con búsqueda exhaustiva de hiperparámetros (GridSearchCV), evalúa el resultado y guarda el modelo entrenado.

Pipeline de preprocesamiento + modelo

```
X (54 features) → StandardScaler → SVC
```

Por qué StandardScaler antes del SVM:

El SVM maximiza el margen entre clases en el espacio de características. Este margen depende de distancias euclidianas, por lo que es sensible a la escala. Si una feature tiene rango [0, 1000] y otra [0, 1], la primera domina el cálculo del margen aunque no sea más informativa. StandardScaler transforma cada feature a:

$$z = \frac{x - \mu}{\sigma}$$

con μ y σ calculados **solo sobre el conjunto de entrenamiento** (y aplicados al test sin recalcular), evitando data leakage.

Validación cruzada estratificada — StratifiedKFold

Qué hace: divide el conjunto de entrenamiento en `k` folds manteniendo la misma proporción de clases en cada fold.

Por qué estratificado:

Con datasets pequeños y desbalanceados (lo habitual en narices electrónicas de laboratorio), un fold aleatorio podría quedarse sin ejemplos de una clase, haciendo imposible el entrenamiento o produciendo métricas engañosas. StratifiedKFold garantiza que cada fold tenga representación proporcional de todas las clases.

Ajuste dinámico de k: el código calcula:

$$k = \min(k_{config}, n_{min_clase})$$

donde n_{min_clase} es el número de muestras de la clase más pequeña en el set de entrenamiento. Esto evita que GridSearchCV falle si hay muy pocas muestras de alguna clase.

Espacio de búsqueda de hiperparámetros (GridSearchCV)

Se prueban dos kernels:

Kernel lineal: $K(x_i, x_j) = x_i \cdot x_j$

Separa clases con un hiperplano. Funciona bien cuando las features ya discriminan linealmente (e.g., con buena extracción). Hiperparámetro: `C` (penalización por violaciones del margen).

Kernel RBF (Gaussian):

$$K(x_i, x_j) = \exp \left(-\gamma \|x_i - x_j\|^2 \right)$$

Mapea los datos implícitamente a un espacio de dimensión infinita donde pueden ser separables aunque no lo sean en el espacio original. Dos hiperparámetros: `C` y `gamma`.

- `C` controla el trade-off entre margen amplio y errores de clasificación: `C` grande = margen más ajustado, menos errores en entrenamiento pero más riesgo de overfitting.
- `gamma` controla el radio de influencia de cada punto: `gamma` grande = cada punto influye solo sobre sus vecinos más cercanos (modelo más local, más riesgo de overfitting).

Combinaciones probadas en la búsqueda completa: 4 (lineal) + 4x4 (rbf) = **20 combinaciones** × k folds evaluaciones.

Métricas de evaluación

Métrica	Qué mide
Accuracy	Proporción de predicciones correctas. Engañosa con clases desbalanceadas.
Precision por clase	De las veces que predijo clase X, ¿cuántas eran realmente X?
Recall por clase	De las muestras reales de clase X, ¿cuántas detectó correctamente?
F1-score	Media armónica de precision y recall. Equilibra ambos.
Matriz de confusión	Tabla completa de predicciones vs. valores reales. Muestra qué clases se confunden entre sí.

La brecha `train_accuracy - test_accuracy` es el indicador principal de overfitting. Si es >10%, el modelo ha memorizado el conjunto de entrenamiento.

Limitación actual: se reporta solo accuracy en la validación cruzada (`scoring='accuracy'`). Con clases desbalanceadas debería usarse `scoring='f1_macro'` o `scoring='balanced_accuracy'` .

config.py — Configuración centralizada

Todos los parámetros del sistema están en un único lugar. Los más relevantes a modificar:

Parámetro	Valor actual	Efecto de cambiarlo
<code>FEATURE_MODE</code>	<code>'pca_signal'</code>	<code>'handcrafted'</code> → V2; <code>'pca_signal'</code> → V3 con PCA
<code>PCA_CONFIG['explained_variance_threshold']</code>	0.95	Más alto → más PCs retenidos, más features
<code>PCA_CONFIG['n_components']</code>	None	Entero → número fijo de PCs (ignora el umbral)
<code>savgol_window</code>	15	Mayor → más suavizado, picos más anchos
<code>savgol_polyorder</code>	3	Mayor → ajuste más fiel a picos, menos suavizado
<code>baseline_seconds</code>	2.0	Más → línea base más estable, menos señal útil
<code>time_windows</code>	[(0,2),(2,10), (10,20)]	Definición de las fases temporales del experimento
<code>n_splits_cv</code>	3	Más folds → evaluación más robusta, más tiempo
<code>GRID_PARAMS</code>	lineal+rbf	Espacio de búsqueda del SVM

Para búsqueda rápida usar `GRID_PARAMS_REDUCED` ; para búsqueda exhaustiva, `GRID_PARAMS_EXTENDED` .

utils.py — Utilidades

Funciones de soporte sin lógica de negocio propia:

- `setup_logging` : configura escritura simultánea a consola y a `logs/enose_project.log` .
- `extract_substance_label` : mapea nombres de archivo a etiquetas de clase mediante búsqueda de substring en mayúsculas. Es el punto más frágil del sistema ante cambios en la convención de nombres.
- `validate_sensor_data` : comprueba que estén las 6 columnas de sensores, que sean numéricas y que no haya NaN ni infinitos.

- **Funciones de visualización** (`plot_class_distribution` , `plot_feature_statistics` , `plot_correlation_heatmap`): generan PNGs en `data/processed/visualizations/` .

Puntos de mejora identificados

Área	Problema	Posible solución
Data leakage en PCA	PCA se ajusta sobre todos los datos antes del split train/test	Integrar PCA dentro del pipeline de Phase 5 como paso previo al scaler
Señal demasiado corta	Se omite el filtro Savitzky-Golay sin marcar el archivo como inválido	Rechazar archivos con menos de <code>savgol_window</code> muestras
Normalización fallback	Si $R_0=0$ se usa Z-score, no equivalente físicamente	Marcar el archivo como corrupto en lugar de continuar
Etiquetado por nombre	Frágil ante variaciones en el nombre del archivo	Añadir un CSV de manifiesto con nombre → clase
Métrica de CV	Se usa accuracy, engañosa con clases desbalanceadas	Cambiar a <code>f1_macro</code> o <code>balanced_accuracy</code>
Mezcla de configuraciones	No hay hash ni versión del dataset	Incluir metadatos de configuración en el CSV o en un archivo sidecar
Ventanas fijas	Las ventanas temporales son globales, no adaptativas por muestra	Detección automática del inicio/fin de la inyección (umbral sobre la derivada)
Etanol como clase	ETH está mezclado con las clases de vino en el mismo clasificador	Considerar un clasificador binario previo (¿es etanol o es vino?) antes del de calidad